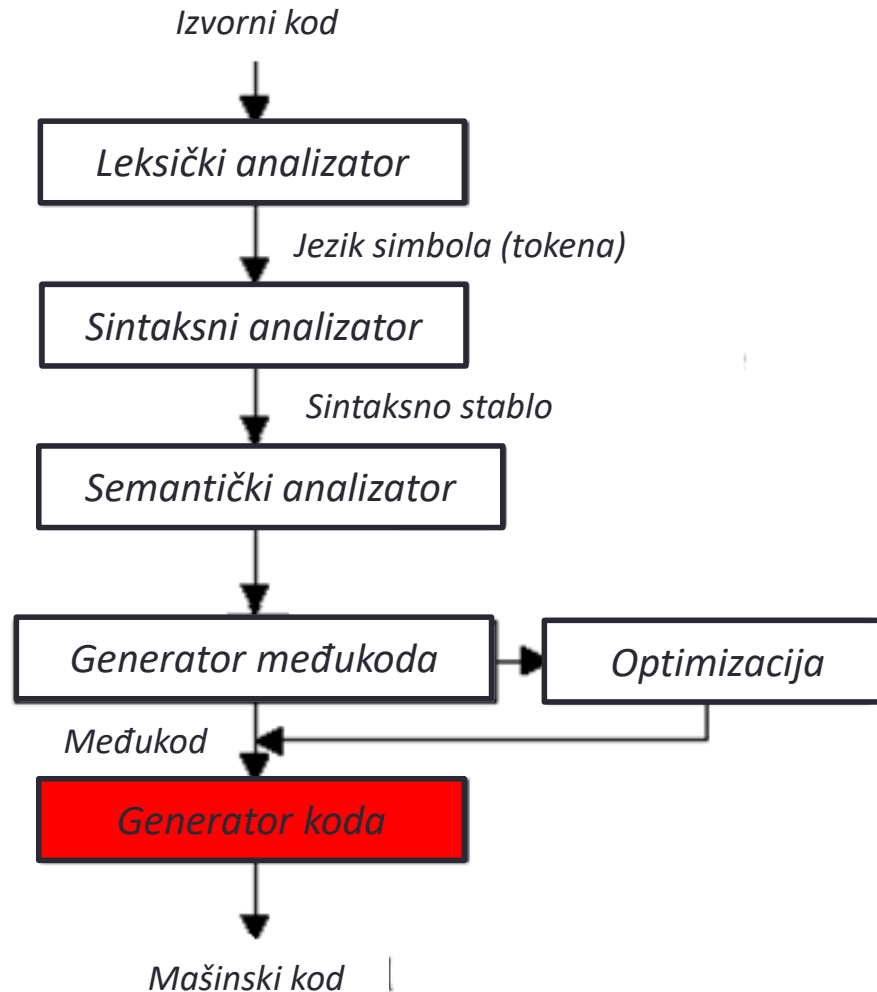


PROGRAMSKI PREVODIOCI

- Generisanje izlaznog koda -**

Struktura kompilatora



Preduslov za razvoj generatora koda

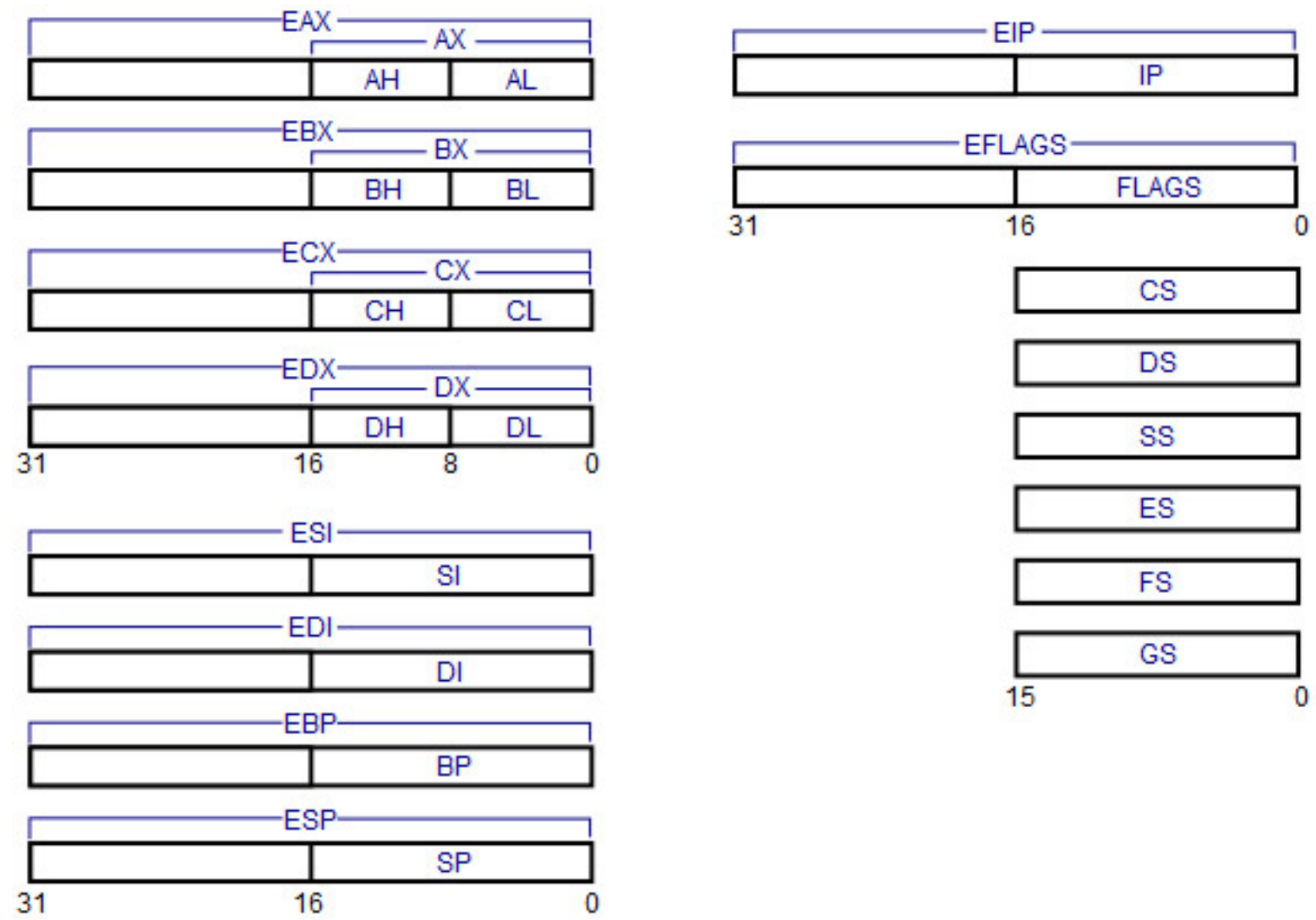
- Poznavanje odredišne mašine za koju se kod generiše
 - Asemblerski jezik,
 - Arhitektura procesora (skup registara i njihove namene)

Zadaci generatora koda

- **Izbor strategije za alokaciju memorije (runtime memory management).**
- Definirati način preslikavanja naredbi međukoda u naredbe asemblerskog jezika.

Odredišna mašina – procesori familije X86-32 (IA-32)

- Registri procesora



Skup instrukcija procesora X86-32

Instrukcija za prenos podataka

MOV <i>dst, src</i>	$[dst] \leftarrow [src]$
---------------------	--------------------------

Aritmetičke instrukcije

ADD <i>dst, src</i>	$[dst] \leftarrow [dst] + [src]$
---------------------	----------------------------------

SUB <i>dst, src</i>	$[dst] \leftarrow [dst] - [src]$
---------------------	----------------------------------

INC <i>src</i>	$[dst] \leftarrow [dst] + 1$
----------------	------------------------------

DEC <i>src</i>	$[dst] \leftarrow [dst] - 1$
----------------	------------------------------

MUL <i>dst, src</i>	$[dst] \leftarrow [dst] * [src]$
---------------------	----------------------------------

DIV <i>src</i>	$[EAX] \leftarrow [EDX : EAX] / [src], [EDX] \leftarrow [EDX : EAX] \bmod [src]$
----------------	--

Logičke instrukcije

AND <i>dst, src</i>	$[dst] \leftarrow [dst] \wedge [src]$
---------------------	---------------------------------------

OR <i>dst, src</i>	$[dst] \leftarrow [dst] \vee [src]$
--------------------	-------------------------------------

XOR <i>dst, src</i>	$[dst] \leftarrow [dst] \oplus [src]$
---------------------	---------------------------------------

NOT <i>src</i>	$[dst] \leftarrow \neg [dst]$
----------------	-------------------------------

Skup instrukcija procesora X86-32

Instrukcija za rad sa stekom

PUSH <i>src</i>	$[stek[ESP]] \leftarrow [src], [ESP] \leftarrow [ESP] - size(src)$
POP <i>dst</i>	$[dst] \leftarrow [stek[SP]], [SP] \leftarrow [SP] + size(dst)$

Instrukcije za promenu toka izvršenja programa

JMP <i>adr</i>	$[EIP] \leftarrow [adr]$
Jc <i>adr</i>	if c then $[EIP] \leftarrow [adr]$
CALL <i>adr</i>	PUSH EIP, $[EIP] \leftarrow [adr]$
RET	POP EIP

Načini adresiranja kod procesora X86-32

Način adresiranja	Format
Neposredno	CONST
Direktno memorijsko	NAME, [CONST]
Direktno registarsko	<i>reg</i>
Indirektno	[<i>reg</i>], [<i>reg</i> +CONST],[<i>reg</i> + <i>reg</i> *CONST], [<i>reg</i> + <i>reg</i> *CONST+CONST]
Indeksno	NAME[<i>reg</i>], CONST[<i>reg</i>], [NAME + <i>reg</i>], [NAME+ <i>reg</i> *CONST],...

Upravljanje memorijom

- Zadaci:
 - Organizacija memorije u toku izvršenja programa:
 - odredjivanje gde će se u memoriji smestiti kod programa,
 - kako će se memorisati podaci različitih tipova,
 - gde će se u memoriji smestiti podaci različitih klasa memorije
 - Pristup podacima smeštenim u različitim zonama memorije
 - Promena sadržaja memorije u trenutku poziva potprograma i povratka na pozivajući modul
 - Prevođenje naredbe poziva i naredbe povratka

Organizacija memorije

- U toku izvršenja programa u memoriji se čuva:
 - Kod programa,
 - Podaci koji se u programu obradjuju
 - Podaci o pozivima potprogramima – aktivacioni slogovi potprograma

Strategije upravljanja memorijom

- **Statička alokacija memorije**

- Sadržaj memorije se ne menja u toku izvršenja programa – i kod programa i svi podaci su poznati u fazi prevodjenja
 - Za svaki potprogram postoji samo jedan aktivacioni slog što znači da ova strategija ne podržava rekurzivne potprograme

- **Dinamička alokacija memorije**

- Sadržaj memorije se stalno menja u toku izvršenja programa
 - mogu se kreirati novi podaci, brisati
 - mogu se kreirati novi delovi koda, brisati
 - mogu se kreirati novi aktivacioni slogovi, brisati
- Povremeno se aktiviraju posebni pomoćni programi (**garbage collector**) koji vode računa o korišćenju memorijskog prostora
 - kad se uoče delovi memorije na koje ne ukazuje ni jedna referenca, oni se uklanjaju iz memorije, tj. taj deo memorije se oslobađa

Strategije upravljanja memorijom

- **Polu-statička ili stek alokacija memorije**
 - Operativna memorija se deli na 2 zone:
 - Statičku – čuva kod programa i globalne podatke
 - Dinamičku koja se deli na:
 - Stack – čuva aktivacione slogove potprograma
 - Heap – čuva podatke koji se po potrebi kreiraju u toku izvršenja programa i uklanjaju kada više nisu potrebni

Struktura aktivacionog sloga

<i>Rezultat potprograma</i>
<i>Stvarni parametri</i>
<i>Link prema aktivacionom slogu pozivajućeg modula</i>
<i>Link prema podacima pozivajućeg modula</i>
<i>Stanje procesora u trenutku poziva</i>
<i>Lokalni podaci</i>
<i>Privrameni podaci</i>

Primer statičke alokacije memorije

PROGRAM CONSUME

```
CHARACTER * 50 BAF
INTEGER NEXT
CHARACTER C, PRODUCE
DATA NEXT /1/, BUF /' '/
6 C = PRODUCE ( )
  BUF(NEXT:NEXT) = C
  NEXT = NEXT + 1
  IF (C .NE. ' ') GO TO 6
  WRITE (*, '(A)') BUF
END
```

CHACTER FUNCTION PRODUCE ()

```
CHARACTER * 80 BUFFER
INTEGER NEXT
SAVE BUFFER, NEXT
DATA NEXT /81/
IF (NEXT .GT. 80 ) THEN
  READ (*, '(A)') BUFFER
  NEXT = 1
END IF
PRODUCE = BUFFER(NEXT:NEXT)
NEXT = NEXT + 1
END
```

Primer statičke alokacije memorije

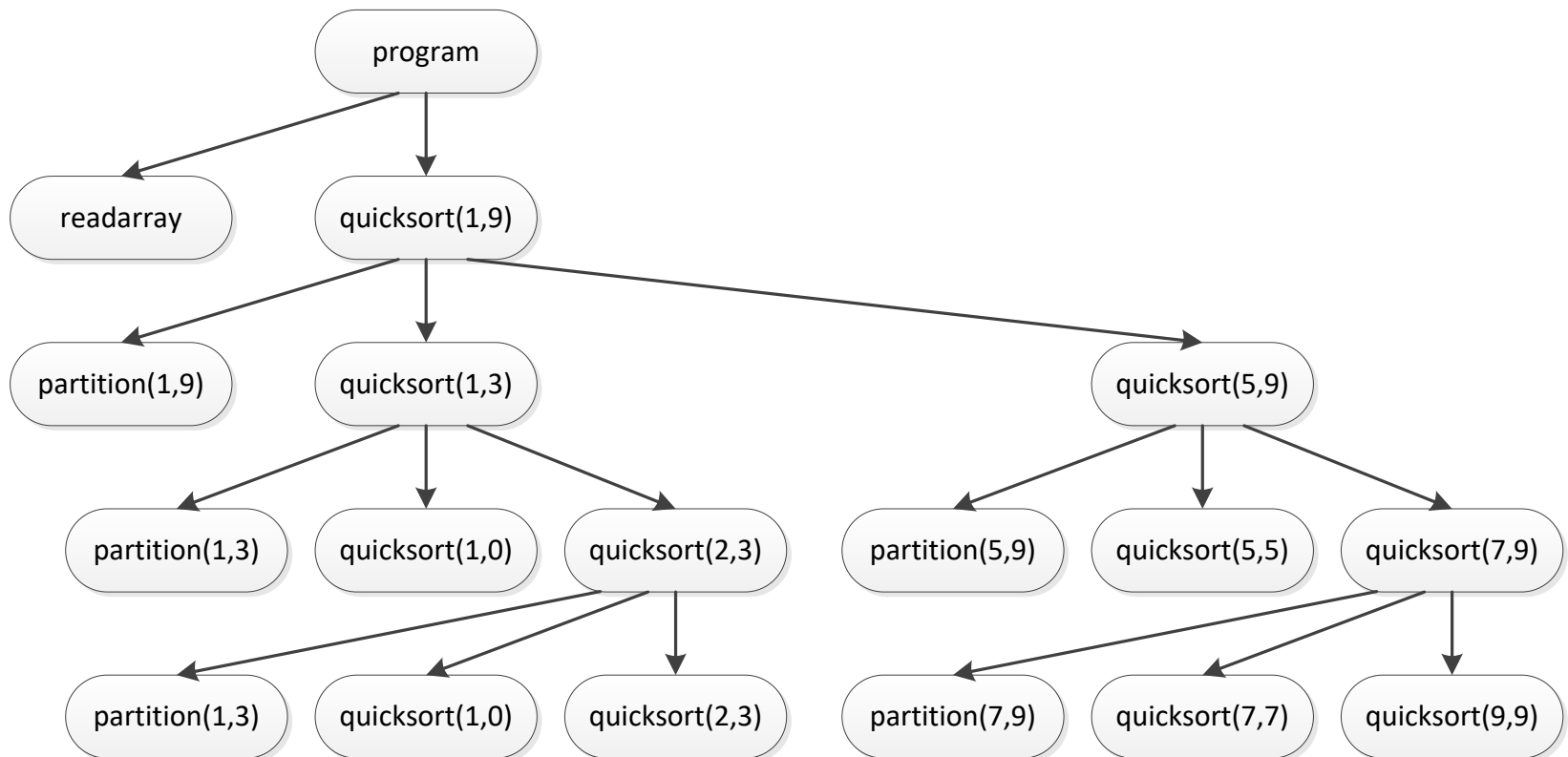
Aktivacioni slog za PRODUCE
Aktivacioni slog za CONSUME
Kod funkcije PRODUCE
Kod programa CONSUME

Primer za stek i dinamičku alokaciju memorije

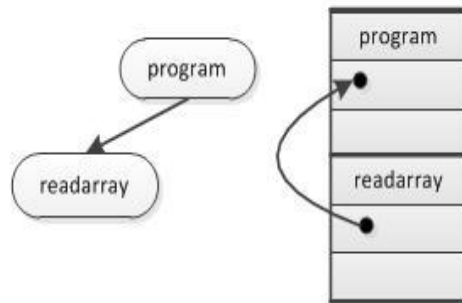
```
int a[11];
void quiksort(int m, int n)
{ int i;
  if (n > m)
  {
    i = partition(m,n);
    quiksort(m, i - 1);
    quiksort(i + 1, n);
  }
}
void main ()
{
  a[0] = -9999;
  a[10] = 9999;
  readarray();

  quiksort(1,9);
}
```

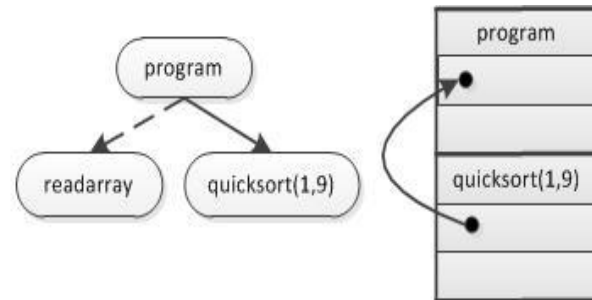

Aktivaciono stablo za program iz primera



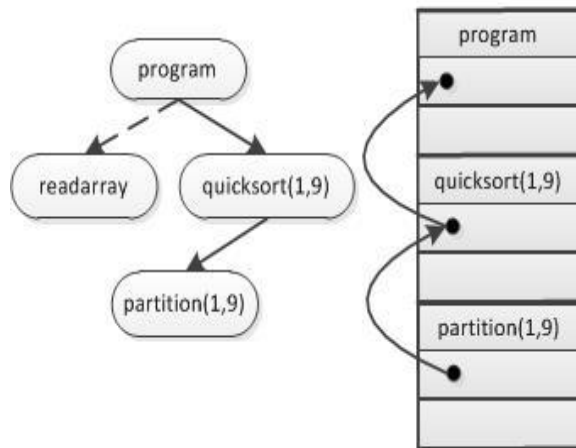
Promena sadržaja memorije kod stek alokacije



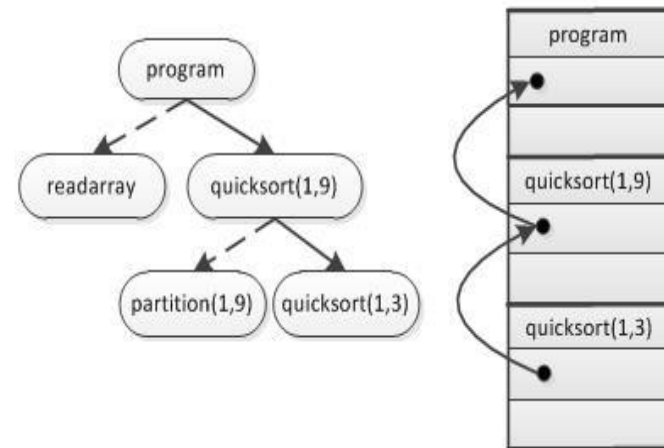
(a)



(b)

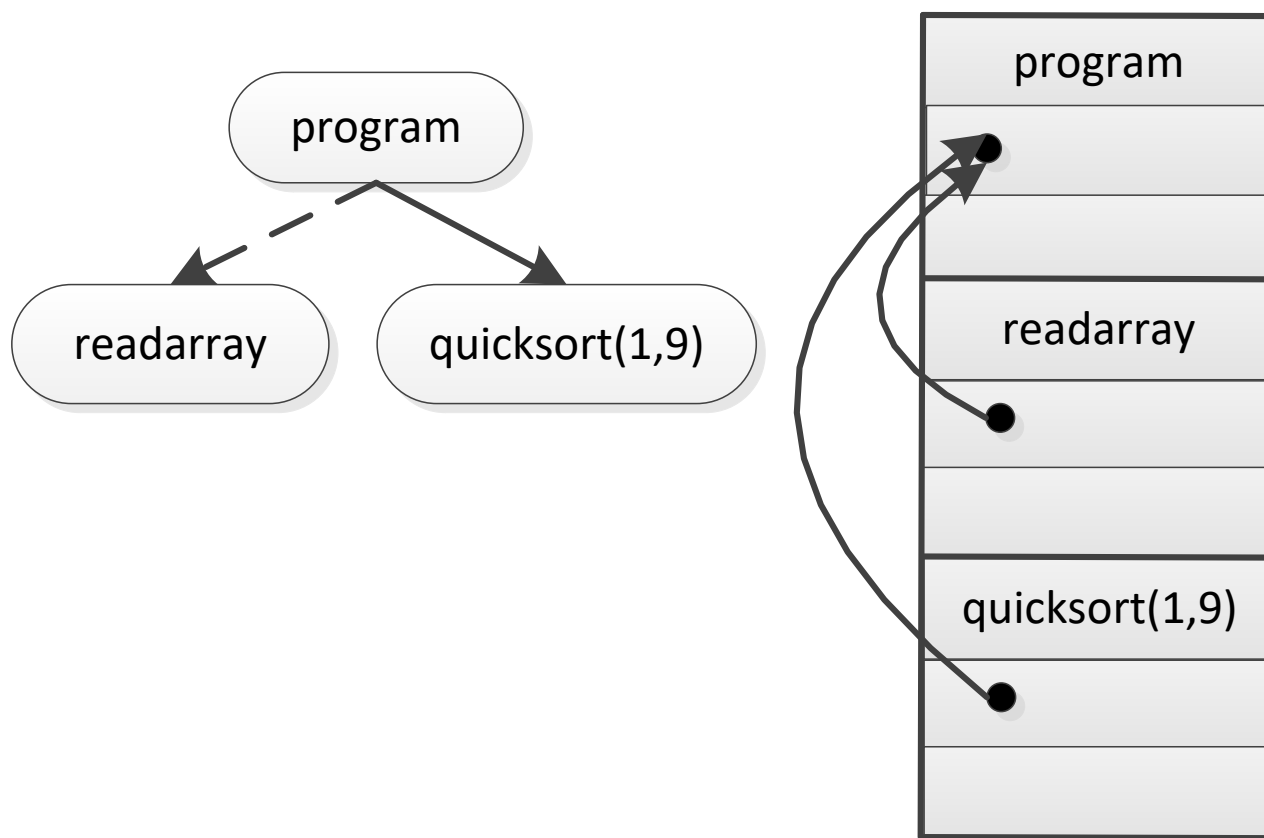


(c)

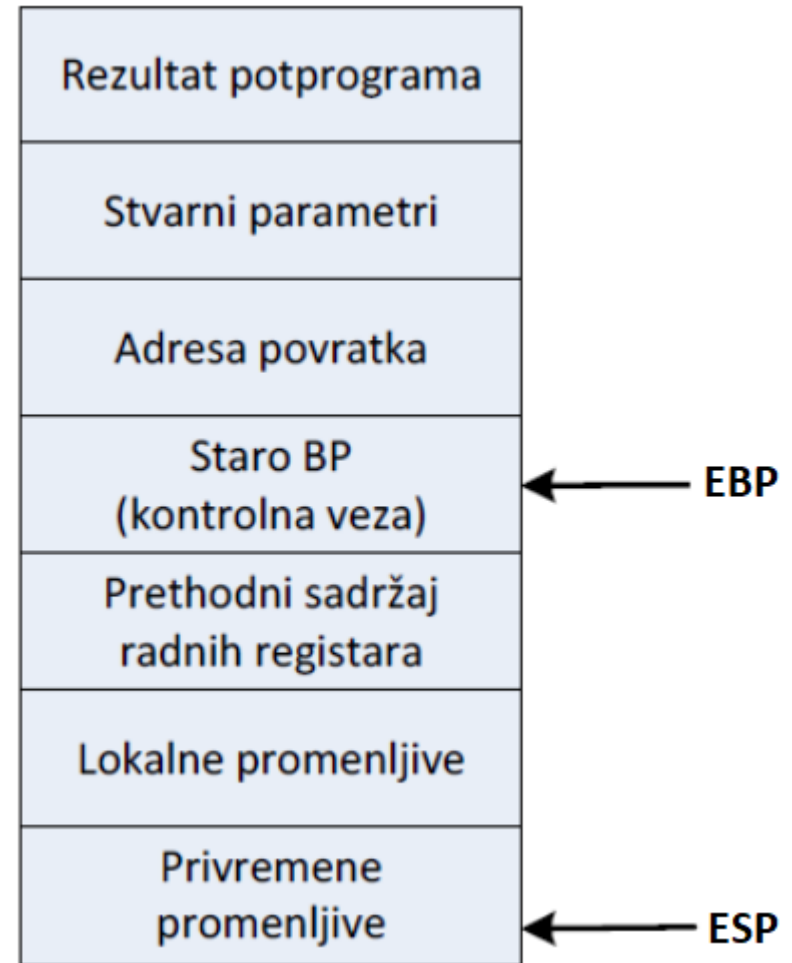
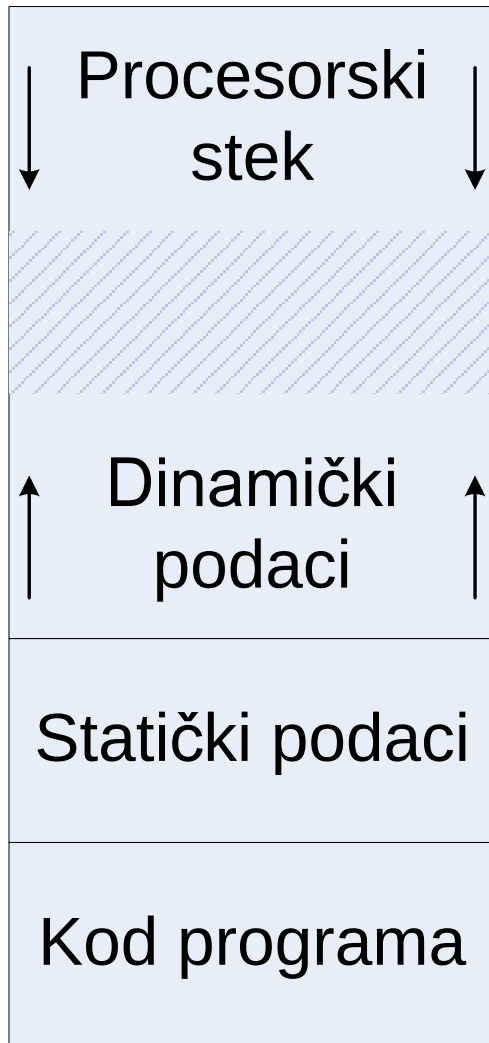


(d)

Promena sadržaja memorije kod dinamičke alokacije



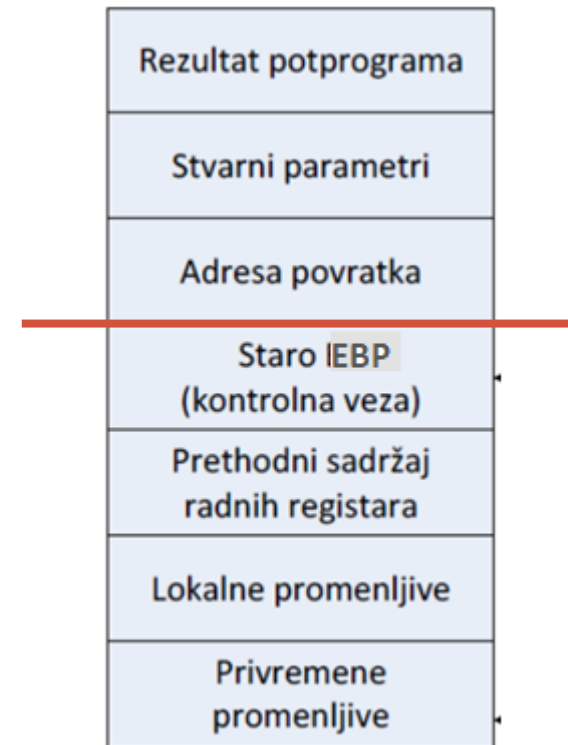
Stek alokacija memorije kod procesora X86-32



Aktivacioni slog u jeziku X86

Generisanje sekvence poziva

- Sekvenca poziva – skup instrukcija koje će se izvršiti u trenutku poziva potprograma
 - Deo instrukcija koje pripadaju pozivajućem modulu
 - Rezervacija prostora za rezultat potprograma
`SUB ESP, size`
 - Smeštanje u stek stvarnih parametara
`PUSH arg_n`
...
`PUSH arg_1`
 - Poziv potprograma
`CALL name`



Generisanje sekvence poziva

- Deo instrukcija koje pripadaju pozvanom modulu
 - Smeštanje u stek prethodne vrednosti baznog pokazivača

PUSH EBP

- Promena sadržaja baznog pokazivača

MOV EBP, ESP

- Smeštanje u stek konteksta procesora
(sadržaja radnih registara)

PUSH EAX

PUSH EBX

PUSH ECX

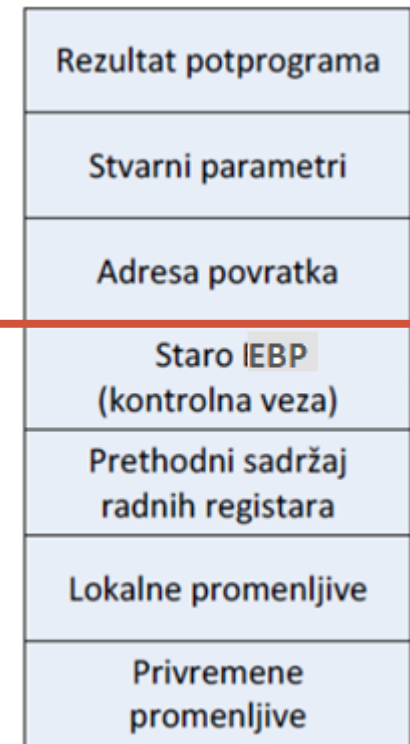
PUSH EDX

PUSH ESI

PUSH EDI

- Rezervacija prostora za lokalne promenljive

SUB ESP, size



Generisana sekvenca poziva za *quicksort(1,9)*

Pozivajući modul

```
MOVE EAX, 9
PUSH EAX
MOVE EAX, 1
PUSH EAX
CALL QUICKSORT
```

Pozvani podul

```
PUSH EBP
MOVE EBP, ESP
PUSH EAX
PUSH EBX
PUSH ECX
PUSH EDX
PUSH ESI
PUSH EDI
SUB ESP, 4
```

Generisanje sekvence povratka

- Sekvenca povratka – skup instrukcija koje će se izvršiti u trenutku povratka u pozivajući modul

- Deo instrukcija koje pripadaju pozvanom modulu

- Upis rezultata u predviđenu lokaciju

```
MOV [EBP+x], rez
```

- Izbacivanje lokalnih promenljivih

```
ADD ESP, size
```

- Vraćanje starog konteksta procesora

```
POP EDI
```

```
POP ESI
```

```
POP EDX
```

```
POP ECX
```

```
POP EBX
```

```
POP EAX
```

- Preuzimanje stare vrednosti baznog pokazivača

```
POP EBP
```

- Povratak u pozivajući modul

```
RET
```


Generisanje sekvence povratka

- Deo instrukcija koje pripadaju pozivajućem modulu
 - Izbacivanje stvarnih parametara is steka
`ADD ESP, size`
 - Preuzivanje rezultata
`POP reg`

Generisana sekvenca povratka za *quicksort(1,9)*

Pozvani podul

ADD ESP, 4

POP EDI

POP ESI

POP EDX

POP ECX

POP EBX

POP EAX

POP EBP

RET

Pozivajući podul

ADD SP, 8

Smeštanje podataka i pristup podacima

- Gde će podaci biti smešteni zavisi od toga gde su definisani i koju klasu memorije imaju.
- U programskom jeziku C:
 - Globalni podaci i lokalni podaci sa klasom memorije static
 - u statičkoj zoni memorije
 - pristup - direktno memorijsko adresiranje (NAME ili [CONST])
 - Parametri funkcija i lokalni podaci sa klasom memorije auto
 - u steku (u aktivacionom slogu)
 - pristup - indirektno adresiranje ([EBP+d])
 - Lokalni podaci sa klasom memorije register
 - u reistrima procesora
 - pristup - direktno registarsko adresiranje (EDX)
 - Podaci u dinamičkoj zoni memorije
 - u heapu
 - pristup - indirektno adresiranje (adresa podatka u registru) ([EBX])

Prevodjenje naredbi međukoda

Prevodjenje naredbe oblika: $x := y \text{ op } z$

Napomena: Za sve registre se uvode deskriptori koji kažu koji podatak oni trenutno sadrže

1. Odrediti lokaciju L gde će biti smešten rezultat x (Ako je L registar, postaviti odgovarajući deskriptor)
2. Ukoliko y nije u L generisati instrukciju. MOV L, y'
 - Ako je L registar, postaviti deskriptor tog registra na x, izbrisati x iz svih ostalih deskriptora
3. Generisati instrukciju: OP L, z'
4. Oslobađanje registara:
 - Ukoliko su y i z u registrima i ne koriste se više u tekućem bloku, osloboditi registre (odgovarajuće deskriptore postaviti na „slobodan“)

Određivanje lokacije rezultata (L)

1. Ako je y u registru, i y se više ne koristi u tekućem bloku, vratiti taj registar kao L.
2. Ako prva opcija nije moguća i postoji slobodan registar, vratiti slobodan registar kao L.
3. Ako operacija OP zahteva da prvi operand bude u nekom konkretnom registru, osloboditi taj registar i vratiti registar kao L.
4. Ako ni jedna od prethodnih opcija nije moguća. vratiti adresu promenljive X kao L.

Prevođenje naredbi dodele - primer

Naredba:

$a = b * (d + c) + b * (d + c)$

Medjukod:

$t1 := d + c$

$t2 := b * t1$

$t3 := t2 + t2$

$a := t3$

Naredba	Generisani medjukod	Deskriptori registara	Deskriptori adresa
		Registri prazni	a, b, c, d u mem.
$t1 := d + c$	MOV EAX, d ADD EAX, c	EAX sadrzi $t1$	$t1$ u EAX a, b, c, d u mem.
$t2 := b * t1$	MOV EBX, b MUL EBX, EAX	EBX sadrzi $t2$	$t2$ u EBX a, b, c, d u mem.
$t3 := t2 + t2$	ADD EBX, EBX	EBX sadrzi $t3$	$t3$ u EBX a, b, c, d u mem.
$a := t3$	MOV a, EBX		a, b, c, d u mem.

Generisanje koda za naredbe sa indeksiranim promenljivama

Naredba medjukoda	Indeks je u registru	Indeks je u memorijskoj lokaciji MI	Indeks je u steku sa pomerajem di u odnosu na sadržaj EBP
$a := b[i]$	MOV EAX, b[ESI] MOV a, EAX	MOV ESI, MI MOV EAX, b[ESI] MOV a, EAX	MOV ESI, [EBP+di] MOV EAX, b[ESI] MOV a, EAX
$b[i] := a$	MOV EAX, a MOV b[EDI], EAX	MOV EDI, MI MOV EAX, a MOV b[SI], EAX	MOV EDI, [EBP+di] MOV EAX, a MOV b[SI], EAX

Generisanje koda za naredbe sa pokazivačem

Naredba medjukoda	Pokazivač je u registru EBX	Pokazivač je u memorijskoj lokaciji MP	Pokazivač je u steku sa pomerajem dp u odnosu na sadržaj EBP
$a := *p$	MOV EAX, [EBX] MOV a, EAX	MOV EBX, MP MOV EAX, [EBX] MOV a, EAX	MOV EBX, [EBP+dp] MOV EAX, [EBX] MOV a, EAX
$*p := a$	MOV EAX, a MOV [EBX], EAX	MOV EBX, MP MOV EAX, a MOV [EBX], EAX	MOV EBX, [EBP+dp] MOV EAX, a MOV [EBX], EAX

Generisanje koda za naredbe uslovnog skoka

`if x relop y goto z`

Generisani kod:

CMP 'x, 'y
JNGE z